

Cấu trúc Project — Website Bán Quần Áo

Nhóm 10 | Môn: Mẫu Thiết Kế Phần Mềm | Cập nhật: 2026-05-31

1. Tổng quan Tech Stack

Layer	Công nghệ	Lý do
Frontend	React + Vite + TypeScript	SPA hiện đại, build nhanh, type-safe
Backend	Spring Boot 3.x + Java 17	Ecosystem mạnh, OOP rõ ràng — phù hợp áp dụng Design Patterns
Database	PostgreSQL 16	Đã thiết kế 23 bảng
Cache	Redis 7	Session, giỏ hàng guest, OTP, cache
Storage	AWS S3	Ảnh sản phẩm, avatar, banner
Containerization	Docker Compose	Đồng nhất môi trường dev/prod

2. Cấu trúc Monorepo

```
clothing-store/
├── docker-compose.yml           # Orchestrate tất cả services
├── docker-compose.dev.yml      # Override cho môi trường dev
├── .env.example                # Template biến môi trường
├── .gitignore
├── README.md
|
├── docs/                       # Tài liệu báo cáo (nộp cho CBGD)
│   ├── BaoCaoCuoiKy.docx       # Báo cáo WORD hoàn chỉnh
│   ├── database_design.md       # Thiết kế CSDL
│   ├── database_schema.sql      # Script tạo DB
│   ├── infrastructure.md        # Kiến trúc hạ tầng
│   ├── diagrams/               # Lược đồ UML
│   │   ├── usecase_overview.xml # Use case tổng quát
│   │   ├── usecase_details/     # Use case chi tiết
│   │   ├── sequence/            # Lược đồ tuần tự (5-10 UC chính)
│   │   └── class/               # Sơ đồ lớp (3-5 UC chính)
│   └── design-patterns/         # Tài liệu Design Patterns
│       ├── README.md            # Tổng hợp các pattern đã dùng
│       ├── strategy-pattern.md   # VD: Strategy cho thanh toán
│       ├── observer-pattern.md   # VD: Observer cho notifications
│       └── factory-pattern.md    # VD: Factory cho voucher
|
├── backend/                    # Spring Boot API
│   ├── Dockerfile
│   ├── pom.xml                 # Maven dependencies
│   └── src/
│       ├── main/
│       │   ├── java/com/nhom10/clothingstore/
│       │   │   ├── ClothingStoreApplication.java
│       │   │   |
│       │   │   ├── config/          # Cấu hình
│       │   │   │   ├── SecurityConfig.java
│       │   │   │   ├── CorsConfig.java
│       │   │   │   ├── RedisConfig.java
│       │   │   │   └── S3Config.java
```

```

| | | └─ entity/                                # JPA Entities (mapping DB)
| | |   └─ User.java
| | |   └─ Product.java
| | |   └─ ProductVariant.java
| | |   └─ Category.java
| | |   └─ Order.java
| | |   └─ OrderItem.java
| | |   └─ Review.java
| | |   └─ Voucher.java
| | |   └─ ...
| | |
| | | └─ enums/                                # Enum types (map với PG
ENUM)
| | |   └─ UserRole.java
| | |   └─ OrderStatus.java
| | |   └─ PaymentMethod.java
| | |   └─ ...
| | |
| | | └─ repository/                          # Spring Data JPA
Repositories
| | |   └─ UserRepository.java
| | |   └─ ProductRepository.java
| | |   └─ OrderRepository.java
| | |   └─ ...
| | |
| | | └─ service/                             # Business Logic
| | |   └─ UserService.java
| | |   └─ ProductService.java
| | |   └─ OrderService.java
| | |   └─ CartService.java
| | |   └─ PaymentService.java
| | |   └─ VoucherService.java
| | |   └─ ReviewService.java
| | |   └─ S3StorageService.java
| | |   └─ EmailService.java
| | |   └─ ...
| | |

```

```
| | | └─ controller/ # REST Controllers
| | | |   └─ AuthController.java
| | | |   └─ ProductController.java
| | | |   └─ OrderController.java
| | | |   └─ CartController.java
| | | |   └─ ReviewController.java
| | | |   └─ admin/ # Admin-only endpoints
| | | | |   └─ AdminProductController.java
| | | | |   └─ AdminOrderController.java
| | | | |   └─ AdminUserController.java
| | | | |   └─ AdminVoucherController.java
| | | | |   └─ AdminBannerController.java
| | | | |   └─ AdminBlogController.java
| | | | |   └─ AdminDashboardController.java
| | | |   └─ ...
| | |
| | | └─ dto/ # Data Transfer Objects
| | | |   └─ request/ # Request DTOs
| | | | |   └─ LoginRequest.java
| | | | |   └─ RegisterRequest.java
| | | | |   └─ CreateOrderRequest.java
| | | | |   └─ ...
| | | |   └─ response/ # Response DTOs
| | | | |   └─ ProductResponse.java
| | | | |   └─ OrderResponse.java
| | | | |   └─ PageResponse.java
| | | | |   └─ ...
| | |
| | | └─ mapper/ # Entity ↔ DTO Mapping
| | | |   └─ ProductMapper.java
| | | |   └─ OrderMapper.java
| | | |   └─ ...
| | |
| | | └─ security/ # JWT + Spring Security
| | | |   └─ JwtTokenProvider.java
| | | |   └─ JwtAuthenticationFilter.java
| | | |   └─ CustomUserDetailsService.java
```

```

| | | | |   └─ OAuth2LoginSuccessHandler.java
| | | | |
| | | | |   └─ exception/                                # Global Exception Handling
| | | | |       └─ GlobalExceptionHandler.java
| | | | |       └─ ResourceNotFoundException.java
| | | | |       └─ BadRequestException.java
| | | | |       └─ UnauthorizedException.java
| | | | |
| | | | |   └─ pattern/                                  # ★ DESIGN PATTERNS
| | | | |       └─ strategy/                            # Strategy Pattern
| | | | |           └─ PaymentStrategy.java              # Interface
| | | | |           └─ CodPaymentStrategy.java           # COD
| | | | |           └─ VnPayPaymentStrategy.java         # VNPay
| | | | |           └─ MoMoPaymentStrategy.java          # MoMo
| | | | |           └─ PaymentContext.java               # Context
| | | | |
| | | | |   └─ observer/                                # Observer Pattern
| | | | |       └─ OrderEvent.java                      # Event
| | | | |       └─ OrderObserver.java                   # Interface
| | | | |       └─ EmailNotificationObserver.java        # Gửi email
| | | | |       └─ InventoryObserver.java               # Cập nhật
| | | | |
| | | | |   └─ LoyaltyPointObserver.java                # Cộng điểm
| | | | |   └─ OrderSubject.java                        # Subject
| | | | |
| | | | |   └─ factory/                                # Factory Method Pattern
| | | | |       └─ DiscountCalculator.java               # Interface
| | | | |       └─ PercentageDiscount.java              # Giảm %
| | | | |       └─ FixedAmountDiscount.java             # Giảm cố'
| | | | |
| | | | |   └─ DiscountFactory.java                     # Factory
| | | | |
| | | | |   └─ builder/                                # Builder Pattern (tùy chọn)
| | | | |       └─ OrderBuilder.java                    # Build đơn
| | | | |
| | | | |   └─ singleton/                              # Singleton (tùy chọn)

```

tồn kho

định

hàng phức tạp

```

├── AppConfig.java # Config
instance
├── aop/ # Aspect-Oriented (Logging)
│   └── ActivityLogAspect.java
├── util/ # Utilities
│   ├── SlugUtils.java
│   ├── OrderCodeGenerator.java
│   └── PaginationUtils.java
├── resources/
│   ├── application.yml # Config chính
│   ├── application-dev.yml # Config dev
│   ├── application-prod.yml # Config prod
│   └── db/
│       ├── migration/ # Flyway migrations
│       │   ├── V1__init_schema.sql
│       │   └── V2__seed_data.sql
├── test/ # Unit + Integration Tests
│   ├── java/com/nhom10/clothingstore/
│   │   ├── service/
│   │   │   ├── OrderServiceTest.java
│   │   │   └── PaymentStrategyTest.java
│   │   └── controller/
│   │       └── ProductControllerTest.java
├── frontend/ # React + Vite
│   ├── Dockerfile
│   ├── package.json
│   ├── vite.config.ts
│   ├── tsconfig.json
│   ├── index.html
│   └── src/
│       ├── main.tsx
│       └── App.tsx

```

```
| | └─ vite-env.d.ts
| |
| | └─ assets/ # Static assets
| |   └─ fonts/
| |   └─ icons/
| |   └─ images/
| |
| | └─ styles/ # Global CSS
| |   └─ index.css # CSS variables, reset
| |   └─ typography.css
| |   └─ animations.css
| |
| | └─ components/ # Reusable UI components
| |   └─ ui/ # Atomic components
| |     └─ Button.tsx
| |     └─ Input.tsx
| |     └─ Modal.tsx
| |     └─ Badge.tsx
| |     └─ Rating.tsx
| |     └─ Pagination.tsx
| |     └─ LoadingSpinner.tsx
| |
| |   └─ layout/ # Layout components
| |     └─ Header.tsx
| |     └─ Footer.tsx
| |     └─ Sidebar.tsx
| |     └─ MainLayout.tsx
| |     └─ AdminLayout.tsx
| |
| |   └─ product/ # Product-specific
| |     └─ ProductCard.tsx
| |     └─ ProductGrid.tsx
| |     └─ ProductFilter.tsx
| |     └─ ProductGallery.tsx
| |     └─ SizeSelector.tsx
| |     └─ ColorSelector.tsx
| |     └─ StickyBuyBar.tsx
```

```
|
|
|   └─ cart/
|       └─ MiniCart.tsx
|       └─ CartItem.tsx
|       └─ CartSummary.tsx
|
|   └─ review/
|       └─ ReviewList.tsx
|       └─ ReviewForm.tsx
|       └─ ReviewFilter.tsx
|
| └─ pages/                                # Route pages
|   └─ Home.tsx
|   └─ ProductList.tsx
|   └─ ProductDetail.tsx
|   └─ Cart.tsx
|   └─ Checkout.tsx
|   └─ OrderHistory.tsx
|   └─ OrderDetail.tsx
|   └─ Wishlist.tsx
|   └─ Profile.tsx
|   └─ Login.tsx
|   └─ Register.tsx
|   └─ Blog.tsx
|   └─ BlogPost.tsx
|   └─ NotFound.tsx
|
|   └─ admin/                            # Admin pages
|       └─ Dashboard.tsx
|       └─ ProductManagement.tsx
|       └─ OrderManagement.tsx
|       └─ UserManagement.tsx
|       └─ VoucherManagement.tsx
|       └─ BannerManagement.tsx
|       └─ BlogManagement.tsx
|       └─ Reports.tsx
|
```

```
├── hooks/                                # Custom React hooks
│   ├── useAuth.ts
│   ├── useCart.ts
│   ├── useProducts.ts
│   ├── useDebounce.ts
│   └── useInfiniteScroll.ts
│
├── services/                            # API calls (Axios)
│   ├── api.ts                          # Axios instance + interceptors
│   ├── authService.ts
│   ├── productService.ts
│   ├── orderService.ts
│   ├── cartService.ts
│   ├── reviewService.ts
│   └── adminService.ts
│
├── store/                              # State management (Zustand)
│   ├── authStore.ts
│   ├── cartStore.ts
│   └── filterStore.ts
│
├── types/                              # TypeScript types
│   ├── product.ts
│   ├── order.ts
│   ├── user.ts
│   └── api.ts
│
├── utils/                              # Helper functions
│   ├── formatPrice.ts
│   ├── formatDate.ts
│   └── validators.ts
│
└── router/                             # React Router config
    ├── index.tsx
    ├── ProtectedRoute.tsx
    └── AdminRoute.tsx
```

```
└─ nginx/  
  └─ Dockerfile  
    └─ nginx.conf
```

```
# Reverse Proxy
```

3. Ảnh xạ Design Patterns → Chức năng

Yêu cầu đề bài: Ít nhất **03 mẫu thiết kế**, chỉ rõ áp dụng cho chức năng gì, lợi ích, hạn chế.

3.1. Strategy Pattern — Thanh toán

Chức năng: Xử lý thanh toán đa phương thức (COD, VNPay, MoMo)

PaymentStrategy (Interface)

```
└─ CodPaymentStrategy      → Xử lý COD  
└─ VnPayPaymentStrategy   → Gọi API VNPay  
└─ MoMoPaymentStrategy    → Gọi API MoMo
```

PaymentContext → Chọn strategy tại runtime

	Chi tiết
Lợi ích	Thêm phương thức mới (ZaloPay, Visa) chỉ cần tạo class mới, không sửa code cũ (Open/Closed)
Hạn chế	Tăng số lượng class, client cần biết có bao nhiêu strategy
Package	<code>com.nhom10.clothingstore.pattern.strategy</code>

3.2. Observer Pattern — Xử lý sự kiện đơn hàng

Chức năng: Khi đơn hàng thay đổi trạng thái → trigger nhiều hành động

OrderSubject (Subject)

- └ EmailNotificationObserver → Gửi email xác nhận
- └ InventoryObserver → Giảm/hoàn tiền kho (QĐ6)
- └ LoyaltyPointObserver → Cộng/trừ điểm tích lũy

	Chi tiết
Lợi ích	Loose coupling: thêm observer mới (VD: SMS notification) không ảnh hưởng OrderService
Hạn chế	Debug khó hơn khi có nhiều observer, thứ tự thực thi không đảm bảo
Package	<code>com.nhom10.clothingstore.pattern.observer</code>

3.3. Factory Method Pattern — Tính giảm giá Voucher

Chức năng: Tạo đúng loại calculator dựa trên `discount_type` (percentage / fixed_amount)

DiscountCalculator (Interface)

- └ PercentageDiscount → Giảm theo %
- └ FixedAmountDiscount → Giảm số tiền cố định

DiscountFactory → Tạo calculator từ `discount_type`

	Chi tiết
Lợi ích	Tách logic tạo object khỏi business logic, dễ test từng loại riêng
Hạn chế	Có thể overkill nếu chỉ có 2 loại, nhưng chuẩn bị cho mở rộng (VD: buy X get Y)
Package	<code>com.nhom10.clothingstore.pattern.factory</code>

3.4. Builder Pattern (Bonus) — Tạo đơn hàng

Chức năng: Build đơn hàng phức tạp (nhiều items, address, voucher, shipping fee, payment)

```
Order order = new OrderBuilder()  
    .withUser(user)  
    .withItems(cartItems)  
    .withAddress(address)  
    .withVoucher(voucher)  
    .calculateShipping()  
    .calculateTotal()  
    .build();
```

	Chi tiết
Lợi ích	Code tạo order rõ ràng, từng bước, dễ đọc, tránh constructor quá nhiều tham số
Hạn chế	Thêm 1 class, nhưng đáng cho object phức tạp như Order

4. Docker Compose

```
# docker-compose.yml
services:
  nginx:
    build: ./nginx
    ports: ["80:80"]
    depends_on: [frontend, backend]

  frontend:
    build: ./frontend
    environment:
      - VITE_API_URL=http://localhost/api

  backend:
    build: ./backend
    ports: ["8080:8080"]
    environment:
      -
    SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/clothing_store
    - SPRING_REDIS_HOST=redis
    - AWS_S3_BUCKET=nhom10-clothing-store-assets
    depends_on: [postgres, redis]

  postgres:
    image: postgres:16-alpine
    ports: ["5432:5432"]
    environment:
      - POSTGRES_DB=clothing_store
      - POSTGRES_USER=nhom10
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./docs/database_schema.sql:/docker-entrypoint-initdb.d/01-
schema.sql

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
```

```
volumes:
  postgres_data:
```

5. Ảnh xạ Use Cases → API Endpoints

5.1. Public APIs (Khách vắng lai + KH)

Method	Endpoint	Use Case	Controller
POST	/api/auth/register	Đăng ký	AuthController
POST	/api/auth/login	Đăng nhập	AuthController
POST	/api/auth/google	Đăng nhập Google	AuthController
POST	/api/auth/forgot-password	Quên mật khẩu	AuthController
GET	/api/products	Xem danh sách SP	ProductController
GET	/api/products/{slug}	Xem chi tiết SP	ProductController
GET	/api/products/search?q=	Tìm kiếm SP	ProductController
GET	/api/categories	Danh mục SP	CategoryController
GET	/api/reviews/{productId}	Xem đánh giá	ReviewController
GET	/api/collections	Bộ sưu tập	CollectionController
GET	/api/blog	Danh sách blog	BlogController
GET	/api/banners	Banner trang chủ	BannerController

5.2. Authenticated APIs (KH đã đăng nhập)

Method	Endpoint	Use Case
GET/PUT	/api/profile	Quản lý thông tin cá nhân
GET/POST/DELETE	/api/addresses	Quản lý địa chỉ

GET/POST/PUT/DELETE	/api/cart	Giỏ hàng
POST	/api/orders	Đặt hàng
GET	/api/orders	Lịch sử đơn hàng
PUT	/api/orders/{id}/cancel	Hủy đơn
POST	/api/reviews	Đánh giá SP
GET/POST/DELETE	/api/wishlists	Yêu thích
POST	/api/payments/vnpay	Thanh toán VNPay
GET	/api/payments/vnpay/callback	VNPay IPN

5.3. Admin APIs

Method	Endpoint	Use Case
CRUD	/api/admin/products	Quản lý sản phẩm
CRUD	/api/admin/categories	Quản lý danh mục
CRUD	/api/admin/orders	Quản lý đơn hàng
CRUD	/api/admin/vouchers	Quản lý voucher
CRUD	/api/admin/banners	Quản lý banner
CRUD	/api/admin/blog	Quản lý blog
GET	/api/admin/users	Quản lý KH
GET	/api/admin/reports/revenue	Thống kê doanh thu (BM1)
GET	/api/admin/reports/bestsellers	SP bán chạy (BM2)
GET	/api/admin/reports/inventory	Tồn kho (BM3)
GET	/api/admin/reports/customers	KH thân thiết (BM5)

6. Lược đồ tuần tự – 5 Use Cases chính (gợi ý)

Đề bài yêu cầu 5-10 lược đồ tuần tự. Gợi ý 5 UC đại diện nhất:

#	Use Case	Lý do chọn	Patterns liên quan
1	Đặt hàng & Thanh toán	UC phức tạp nhất, nhiều bước	Strategy, Builder, Observer
2	Tìm kiếm & Lọc sản phẩm	UC phổ biến nhất, demo pg_trgm	—
3	Đăng ký & Đăng nhập	UC nền tảng, demo OAuth + JWT	—
4	Đánh giá sản phẩm	UC có nhiều business rules (QĐ9, QĐ13)	—
5	Áp dụng Voucher	Demo Factory Pattern	Factory

7. Sơ đồ lớp – 3 UC chính (gợi ý)

Đề bài yêu cầu sơ đồ lớp cho 3-5 UC chính. Gợi ý 3 UC có kiến trúc rõ nhất:

#	Use Case	Các lớp chính
1	Thanh toán	PaymentStrategy , CodPayment , VnPayPayment , MoMoPayment , PaymentContext , PaymentService , PaymentController
2	Xử lý đơn hàng	OrderSubject , OrderObserver , EmailObserver , InventoryObserver , LoyaltyObserver , OrderService , OrderBuilder
3	Giảm giá Voucher	DiscountCalculator , PercentageDiscount , FixedAmountDiscount , DiscountFactory , VoucherService

8. Thứ tự triển khai đề xuất

Phase 1: Foundation (Tuần 1)

- └─ Setup Docker Compose (postgres, redis)
- └─ Spring Boot skeleton + JPA Entities
- └─ Chạy database_schema.sql
- └─ React + Vite skeleton + routing

Phase 2: Core Features (Tuần 2-3)

- └─ Auth (đăng ký, đăng nhập, JWT, Google OAuth)
- └─ Products CRUD (admin) + listing (public)
- └─ Category management
- └─ Product search + filter + sort
- └─ Product detail page

Phase 3: E-commerce (Tuần 3-4)

- └─ Cart (Redis cho guest, DB cho KH)
- └─ Order flow + Builder Pattern
- └─ Payment Strategy Pattern (COD, VNPay sandbox)
- └─ Order Observer Pattern (email, inventory, points)
- └─ Voucher + Factory Pattern

Phase 4: Engagement (Tuần 4-5)

- └─ Reviews + images (S3 upload)
- └─ Wishlist
- └─ Membership tiers
- └─ Banner management
- └─ Blog CRUD

Phase 5: Admin & Reports (Tuần 5-6)

- └─ Admin Dashboard
- └─ Reports (BM1-BM5)
- └─ Activity Logs
- └─ User management

Phase 6: Polish (Tuần 6)

- └─ Responsive design
- └─ Loading states + error handling
- └─ SEO (meta tags, sitemap)

└─ Video demo 5 phút

└─ Báo cáo Word